Development Plan: Recovering the Ifpack2 User-Partitioner Camellia Additive Schwarz Path

Nathan V. Roberts / AI-assisted implementation plan

May 26, 2026

1 Objective

The objective of this effort is to recover and complete the committed branch direction for `CAMELLIA_ADDITIVE_SCHW` in the Tpetra/Ifpack2 migration, while staying as close as practical to the existing committed design. The intended production behavior remains:

- one additive Schwarz subdomain per owned cell,
- each subdomain containing that cell plus its minimal halo/overlap region,
- implementation based on stock Ifpack2 infrastructure where possible,
- initial priority on validating and fixing the committed `PreconditioningTests.cpp` path before re-enabling GMG-level use.

2 Current Understanding

The committed branch does *not* primarily center on the uncommitted `Camellia_AdditiveSchwarz_decl.hpp` and `Camellia_AdditiveSchwarz_def.hpp` prototype. Instead, the main branch direction is encoded in:

A practical dependency note is important here: the currently working `Ifpack2::OverlappingRowMatrix` used by this effort is not yet in main Trilinos, but rather in Nathan V. Roberts' Trilinos fork:

- repository: `https://github.com/CamelliaDPG/Trilinos.git`
- branch: `fixes-for-camellia`

This does not block merge into the target Camellia branch, because that branch already relies on changes from the same Trilinos fork/branch.

- `src/Solution/GMGOperator.cpp`,
- `src/include/GMGOperator.h`,
- `unit_tests/PreconditioningTests.cpp`,
- `unit_tests/GMGSolverTests.cpp`,
- `unit_tests/GMGOperatorTests.cpp`,

- and supporting Tpetra migration code such as `OverlappingRowMatrix` and `Tpetra_Operator_to_Tpetra_Ma`

The committed design in `GMGOperator.cpp` is:

1. compute a Camellia-specific overlapping row set for this rank,
2. compute a user-specified collection of local parts, one per owned cell,
3. instantiate stock Ifpack2 Schwarz,
4. provide:
   - `"schwarz: overlap elements"`,
   - `"partitioner: type" = "user"`,
   - `"partitioner: parts" = partsLocalOrdinals`,
5. and then select an inner preconditioner for the Schwarz-local problem.

A major result of the recovery work is that the outer overlap+user-partitioner mechanism itself is viable. However, the final branch-wide choice of inner solver is still not fully stabilized. In particular, duplicated inner use of the same user partition was shown to be harmful, `Block Relaxation` remains the best observed choice for the targeted problem-level additive Schwarz preconditioning tests, but that same choice still crashes in at least one small multirank `GMGSolver` identity case.

The existing production code does *not* yet re-enable the full Camellia additive Schwarz path, because the diagonal/incidence weighting section currently throws an exception indicating that the Ifpack2 version is still unimplemented.

3 Primary Technical Hypothesis

The leading hypothesis is that the current mechanism based on:

- explicit overlap rows via `"schwarz: overlap elements"`, and
- user partition data via `"partitioner: parts"`

is only partially appropriate for the desired Camellia semantics.

More specifically, the likely issue is that Camellia wants *overlapping element-centered subdomains*, whereas the Ifpack2 user partitioner / block-relaxation interface may fundamentally expect a *partition* of local rows, or at least semantics closer to disjoint block ownership than to additive Schwarz subdomains that reuse the same local row in multiple blocks.

If this hypothesis is correct, then the current committed direction remains conceptually close to the target but is mechanically mismatched at the block-definition level.

4 Evidence Supporting the Hypothesis

The strongest evidence comes from `unit_tests/PreconditioningTests.cpp`.

4.1 Toy custom Schwarz test

`testCustomAdditiveSchwarz(...)` constructs a hand-checkable matrix and manually computes the expected additive Schwarz action, then compares that against the Ifpack2-based operator. This indicates that the branch was already focused on validating operator semantics, not merely successful construction.

4.2 Parallel/serial agreement test

`testCustomAdditiveSchwarzBiggerOverlapParallelSerialAgreement2Ranks(...)` contains an explicit comment describing an MPI-only pathology in which the first row of the apparent inverse operator becomes populated incorrectly in parallel. This strongly suggests that the branch hit a semantic mismatch in overlap/block treatment rather than a simple API wiring bug.

4.3 GMG-level disablement

`unit_tests/GMGSolverTests.cpp` explicitly disables `CAMELLIA_ADDITIVE_SCHWARZ` in the SPD/smoother test loop, which is consistent with a partially completed or currently incorrect implementation path.

5 Constraints

This implementation effort should obey the following constraints.

1. Stay close to the committed branch design if at all possible.
2. Treat the uncommitted `Camellia_AdditiveSchwarz_*` files as exploratory scratch, not the primary implementation path.
3. Prioritize existing `PreconditioningTests.cpp` validation before re-enabling GMG smoother tests.
4. Use Ifpack2/Tpetra-native mechanisms where they are actually compatible with the target semantics.
5. Avoid introducing a large architectural pivot unless the existing tests and code strongly force that conclusion.

6 Implementation Strategy

The work should proceed in phases.

6.1 Phase 1: Validate the current mechanism at the operator-test level

The first task is to determine whether the committed user-partitioner path can, in principle, represent the desired overlapping element-centered Schwarz action.

This phase should focus on:

- understanding the exact semantics of `Ifpack2::AdditiveSchwarz` for:
  - `"schwarz: overlap elements"`,
  - `"partitioner: parts"`,
  - and interaction with `Block Relaxation`,
- inspecting the toy tests in `PreconditioningTests.cpp`,
- identifying whether the current code is using the inner partitioner in a way that duplicates or conflicts with outer Schwarz behavior,

- determining whether the supplied parts are assumed to be disjoint, overlapping, or otherwise constrained by Ifpack2 internals.

Expected outcome of Phase 1:

- either the mechanism is validated as fundamentally sound and only needs a corrective implementation fix,
- or the mechanism is shown to be semantically insufficient for overlapping cell-centered subdomains.

6.2 Phase 2: Make the toy/operator tests authoritative

Before changing GMG production behavior further, tighten the dedicated preconditioning tests so they clearly establish the intended semantics.

This phase may include:

- improving diagnostics in the existing custom additive Schwarz tests,
- adding explicit checks for whether the user parts overlap in local ordinal space,
- comparing serial and parallel operator actions on targeted basis vectors,
- possibly instrumenting overlap-map and parts construction to print or verify exact row reuse patterns.

The goal is that these tests become the decisive signal for whether the current Ifpack2 strategy is semantically correct.

6.3 Phase 3A: If the current mechanism is sound, repair it locally

If Phase 1 shows that the committed approach should work in principle, then repair the implementation while preserving the current architecture.

Likely work items include:

- verifying that the local ordinal parts are computed on the exact overlap map used by the Ifpack2 Schwarz instance,
- checking whether the current explicit overlap matrix helper construction matches the one used internally by Ifpack2,
- ensuring the inner preconditioner parameters are not unintentionally redefining block semantics in a conflicting way,
- validating combine-mode behavior and whether `Add` is actually the correct production mode for the desired operator.

6.4 Phase 3B: If the current mechanism is not sound, apply the smallest viable pivot

If Phase 1 shows that `"partitioner: parts"` cannot faithfully represent overlapping cell-centered subdomains, then the smallest practical pivot should be chosen.

The preferred pivot should still remain close to the committed branch direction. Candidates include:

- continuing to use explicit overlap row construction,
- but ceasing to rely on user partitioning to encode overlapping subdomains,
- and instead introducing a thin Camellia-side layer that restores correct overlapping-subdomain application semantics while still using Tpetra/Ifpack2 building blocks where possible.

This would still be preferable to reviving the uncommitted scratch prototype wholesale, unless that becomes unavoidable.

7 Production Integration Plan

Assuming Phase 1–3 succeed, the next steps are:

1. restore a correct production implementation for the `CAMELLIA_ADDITIVE_SCHWARZ` smoother path in `GMGOperator.cpp`,
2. port or reintroduce the Camellia-specific incidence/multiplicity weighting logic,
3. remove the unconditional exception currently thrown for the diagonal weighting path,
4. re-enable the disabled GMG smoother tests in `GMGSolverTests.cpp`.

8 Specific Files Expected to Change

Primary files likely to change are:

- `src/Solution/GMGOperator.cpp`
- `src/include/GMGOperator.h`
- `unit_tests/PreconditioningTests.cpp`
- `unit_tests/GMGSolverTests.cpp`
- possibly `unit_tests/GMGOperatorTests.cpp`
- possibly `src/include/OverlappingRowMatrix.h` and/or `src/Solution/OverlappingRowMatrix.cpp`
- possibly `src/include/Tpetra_Operator_to_Tpetra_Matrix.h` and `src/MatrixUtilities/Tpetra_Operat` if operator materialization needs improvement for diagnostics.

The uncommitted files:

- `src/include/Camellia_AdditiveSchwarz_decl.hpp`
- `src/include/Camellia_AdditiveSchwarz_def.hpp`

should remain out of scope for the initial implementation unless the committed path is shown to be unsalvageable.

9 Validation Strategy

Validation should proceed in increasing scope.

9.1 Step 1: Toy/operator tests

Focus first on:

- `Preconditioning_CustomAdditiveSchwarzSerial`
- `Preconditioning_CustomAdditiveSchwarzMPI`
- `Preconditioning_CustomAdditiveSchwarzBiggerOverlapParallelSerialAgreement2Ranks`

These should become the first gate for correctness.

9.2 Step 2: Problem-level smoother agreement

Then validate:

- the problem-level preconditioning tests that use `CAMELLIA_ADDITIVE_SCHWARZ`,
- serial/parallel smoother comparison behavior,
- operator extraction behavior where used.

9.3 Step 3: GMG re-enablement

Finally:

- re-enable the `CAMELLIA_ADDITIVE_SCHWARZ` entries in `GMGSolverTests.cpp`,
- verify GMG SPD expectations where applicable,
- confirm that production smoother setup no longer throws the current unimplemented exception.

10 Known Risks

- The outer Ifpack2 overlap+user-partitioner mechanism appears viable, but the final branch-wide inner-solver choice for the Schwarz-local problem is still unresolved.
- `Block Relaxation` remains fragile in at least one small multirank `GMGSolver` identity case, even after removing duplicated inner user partitioning.
- A more solver-like inner choice such as `ILUT` may avoid some crashes, but can substantially weaken the problem-level additive Schwarz preconditioning behavior.
- The current Tpetra operator-to-matrix helper is effectively a stub and may need real implementation for diagnostics.
- Reintroducing incidence-based diagonal weighting may require additional overlap bookkeeping beyond what the current committed path preserves.

11 Bounded Unknowns

The most important remaining bounded unknown is:

> What is the robust branch-wide inner-solver policy for the Schwarz-local problem, once the outer Ifpack2 overlap+user-partitioner mechanism has been accepted as viable?

The work so far has substantially narrowed the uncertainty. The outer overlap/user-partitioner path has passing focused custom MPI tests, and a direct overlap-membership test now verifies that the produced parts match the intended halo dof sets in the tested case. The remaining uncertainty is therefore no longer about whether the outer mechanism can represent the intended subdomains, but about which inner solver policy is robust across both problem-level preconditioning tests and small multirank `GMGSolver` identity cases.

12 Stop-and-Ask Conditions

Implementation should stop and ask for direction only if one of the following occurs:

1. additional project files become necessary and are not available,
2. the smallest viable fix requires a significant architectural deviation from the committed branch direction.

Otherwise, implementation should proceed autonomously.

13 Immediate Next Action

The immediate next action is to work from the committed `PreconditioningTests.cpp` path and determine whether the current use of:

- explicit overlap rows, and
- `"partitioner: parts"`

can correctly encode overlapping cell-centered Schwarz subdomains.

That answer should drive the rest of the implementation.